

Configuración del Entorno de Desarrollo y Repositorio GitHub

Actividad N°: 16 **Fecha:** 22/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Dejar preparado el entorno de desarrollo local y confirmar el flujo de trabajo sobre el repositorio del proyecto en GitHub, como base para iniciar el desarrollo de los módulos de autenticación, usuarios y control de acceso.

2. Repositorio del proyecto

El proyecto se gestiona en un repositorio GitHub ya creado para FeelTheTickets (Canje FTT). El flujo de trabajo adoptado es:

- Commits descriptivos por cambio funcional (no por archivo suelto).
- Uso de `.gitignore` ya definido en el proyecto para excluir `node_modules/`, archivos `.env`, `dist/`, logs y artefactos temporales — evitando exponer credenciales o dependencias pesadas en el repositorio.
- Variables sensibles (`MONGODB_URI`, `JWT_SECRET`, `DEFAULT_PASSWORD`) manejadas exclusivamente vía archivos `.env` locales (no versionados) y variables de entorno del proveedor de despliegue (Render) en producción.

3. Entorno de desarrollo local

Backend

```
cd backend
npm install
cp .env.example .env
# Configurar: MONGODB_URI, JWT_SECRET, DEFAULT_PASSWORD, PORT=5002
npm run dev # nodemon, recarga automática
```

Frontend

```
cd frontend
npm install
# frontend/.env.local → VITE_API_URL=http://localhost:5002
npm run dev # servidor Vite en http://localhost:5173
```

Datos de prueba

```
cd backend
node src/scripts/setup.js ../../LUMINEERS.csv
```

Este comando importa el CSV del evento, extrae las localidades y crea el punto de venta y el usuario administrador inicial (**sistema** / **sistema-inicial**).

4. Verificación del entorno

Verificación	Resultado esperado
GET /health	{ status: 'OK' } — backend arriba
GET /api/health	{ status: 'API_OK' } — API respondiendo con CORS correcto
Conexión a MongoDB Atlas	Log de conexión exitosa en consola/Winston
Frontend en localhost:5173	Pantalla de Login visible con logo FeelTheTickets

5. Convenciones adoptadas para el desarrollo de esta semana

- Los cambios de backend relacionados con autenticación/roles se concentran en **middleware/auth.js**, **controllers/authController.js**, **controllers/userController.js** y los archivos de **routes/**.
- Cualquier ajuste de permisos por rol debe reflejarse en **ambos lados**: el middleware **authorize()** del backend (control real) y el componente **ProtectedRoute**/menú de **Navigation.jsx** del frontend (experiencia de usuario), evitando repetir la inconsistencia detectada en la revisión de diseño de la Semana 3.

6. Conclusiones del día

El entorno de desarrollo queda operativo (backend + frontend + base de datos con datos de prueba) y el flujo de trabajo sobre el repositorio queda confirmado, permitiendo iniciar el desarrollo del módulo de autenticación al día siguiente.

Observaciones: Sin observaciones.

Desarrollo de Autenticación y Acceso al Sistema

Actividad N°: 17 **Fecha:** 23/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

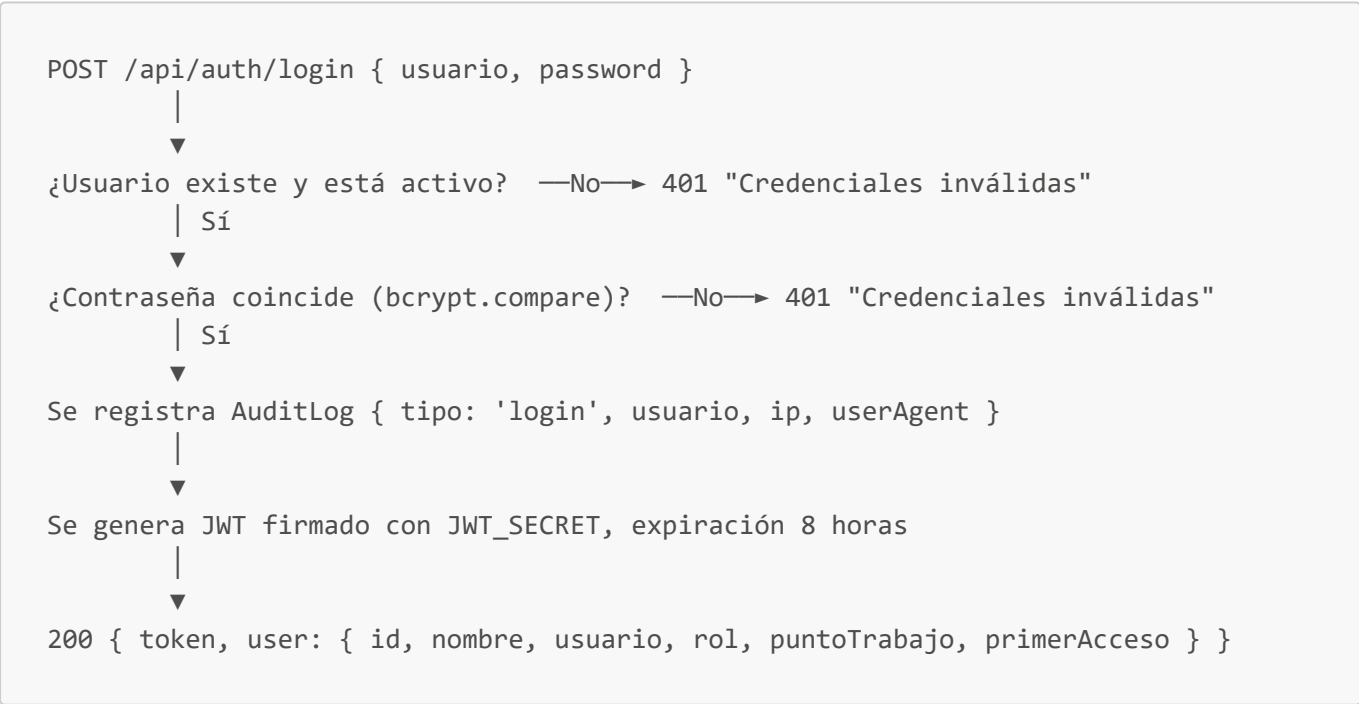
1. Objetivo del día

Implementar y documentar el módulo de autenticación: inicio de sesión, generación y verificación de token, cambio de contraseña en primer acceso, y cierre de sesión.

2. Endpoints implementados (src/routes/auth.js + authController.js)

Método	Ruta	Descripción
POST	/api/auth/login	Autentica usuario y devuelve token JWT
POST	/api/auth/change-password	Cambia la contraseña (requiere sesión)
POST	/api/auth/logout	Registra el cierre de sesión en auditoría
GET	/api/auth/profile	Devuelve los datos del usuario autenticado

3. Flujo de login implementado



Decisión de diseño relevante: el mensaje de error es idéntico ("Credenciales inválidas") tanto si el usuario no existe como si la contraseña es incorrecta, evitando que un atacante pueda enumerar usuarios válidos por diferencia de respuesta.

4. Flujo de cambio de contraseña (incluye primer acceso)

```

POST /api/auth/change-password { currentPassword?, newPassword }
    |
    ▼
¿primerAcceso === true?
├─ Sí → no exige currentPassword (primer cambio obligatorio)
├─ No → exige y valida currentPassword contra el hash almacenado
    |
    ▼
Nueva contraseña ≥ 6 caracteres → se reasigna (hook pre-save la hashea)
    |
    ▼
primerAcceso pasa a false
    |
    ▼
Se registra AuditLog { tipo: 'cambio_password' }

```

Esto asegura que todo usuario nuevo (creado con la contraseña por defecto `DEFAULT_PASSWORD`) esté obligado a definir una contraseña propia antes de operar con normalidad.

5. Middleware de verificación de sesión (`middleware/auth.js`)

- Extrae el token del header `Authorization: Bearer <token>`.
- Verifica la firma y expiración con `jwt.verify`.
- Recupera el usuario en base de datos (excluyendo `password`) y valida que `activo === true`.
- Adjunta `req.user` para que esté disponible en controladores y en los middlewares de autorización/auditoría siguientes.

6. Frontend — `AuthContext`

- Centraliza el estado de sesión (usuario, token) y expone `login()`, `logout()`, `isAuthenticated`.
- Persiste el token para mantener la sesión entre recargas de página.
- Tras un login exitoso, `RoleBasedRedirect` decide la pantalla de destino según el rol (`jefe` → `/dashboard`, `staff` → `/tickets`).

7. Conclusiones del día

El módulo de autenticación queda funcional de extremo a extremo: login seguro sin filtración de información sobre qué credencial falló, manejo obligatorio de contraseña en primer acceso, verificación de sesión vía JWT en cada request protegido, y trazabilidad de login/logout en auditoría.

Observaciones: Sin observaciones.

Implementación de Gestión de Usuarios

Actividad N°: 18 **Fecha:** 24/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar el módulo CRUD de usuarios, restringido al rol Jefe, aplicando las reglas de negocio definidas en la etapa de requerimientos (RF-09).

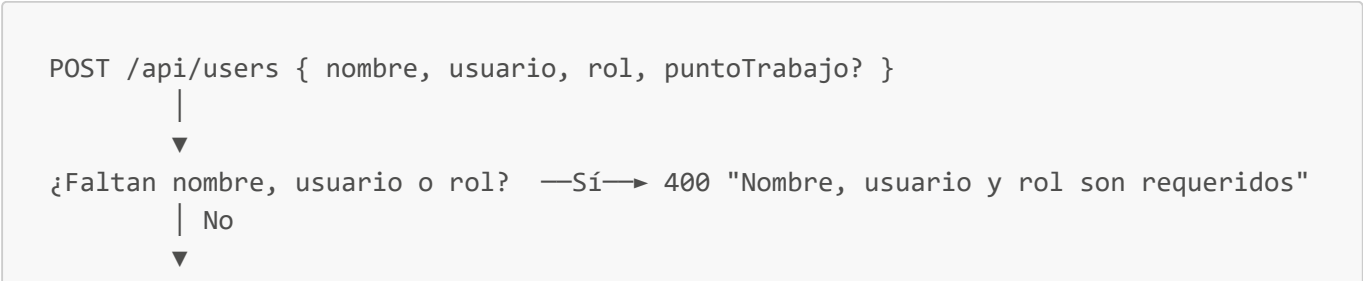
2. Endpoints implementados (src/routes/users.js + userController.js)

Método	Ruta	Acceso	Descripción
POST	/api/users	Jefe	Crear usuario nuevo
GET	/api/users	Jefe	Listar usuarios activos
PUT	/api/users/:id	Jefe	Actualizar usuario
DELETE	/api/users/:id	Jefe	Desactivar usuario (borrado lógico)

3. Reglas de negocio implementadas

- Contraseña inicial estándar:** todo usuario nuevo se crea con la contraseña definida en `DEFAULT_PASSWORD` y `primerAcceso: true`, forzando el cambio en el primer login (conecta con el módulo de autenticación del día anterior).
- Punto de trabajo condicional:** obligatorio si el rol es `staff`, no aplica si el rol es `jefe`.
- Unicidad de usuario:** se verifica explícitamente que el nombre de usuario no exista antes de crear la cuenta, devolviendo un error claro en caso de duplicado.
- Borrado lógico, no físico:** `deleteUser` no elimina el documento, marca `activo: false`, preservando el historial de auditoría asociado a ese usuario (los `AuditLog` referencian `usuario` por `_id`, y borrar el usuario físicamente rompería esa trazabilidad).
- Protección de la propia cuenta:** un usuario Jefe no puede desactivarse ni eliminarse a sí mismo, evitando bloqueos accidentales del único administrador activo.
- Listado solo de activos:** `getUsers` filtra `activo: true` para no mostrar cuentas dadas de baja en la gestión diaria.

4. Diagrama de creación de usuario



```
¿rol = staff y falta puntoTrabajo? —Sí→ 400 "Punto de trabajo es requerido"
  | No
  ▼
¿Usuario ya existe? —Sí→ 400 "El usuario ya existe"
  | No
  ▼
Crear con password = DEFAULT_PASSWORD, primerAcceso = true, creadoPor =
req.user._id
  |
  ▼
201 { usuario creado } (el hook pre-save hashea la contraseña automáticamente)
```

5. Frontend — `UsersPage.jsx`

- Tabla de usuarios activos con nombre, usuario, rol y punto de trabajo.
- Formulario de alta que muestra/oculta el campo "Punto de trabajo" dinámicamente según el rol seleccionado.
- Acciones de editar y desactivar, con confirmación previa (SweetAlert2) antes de desactivar una cuenta.

6. Conclusiones del día

El módulo de gestión de usuarios queda implementado con las validaciones de negocio necesarias (unicidad, punto de trabajo condicional, protección de autoeliminación) y con borrado lógico para preservar la integridad del historial de auditoría.

Observaciones: Sin observaciones.

Implementación de Roles y Permisos

Actividad N°: 19 **Fecha:** 25/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar y consolidar el control de acceso basado en roles (RBAC) en todos los endpoints del sistema, y corregir la inconsistencia de permisos detectada en la revisión de diseño de la Semana 3.

2. Mecanismo de autorización (middleware/auth.js)

```
const authorize = (...roles) => (req, res, next) => {
  if (!req.user) return res.status(401).json({ success: false, message: 'Usuario no autenticado' });
  const flatRoles = roles.flat();
  if (!flatRoles.includes(req.user.rol)) {
    return res.status(403).json({ success: false, message: 'No tiene permisos para realizar esta acción' });
  }
  next();
};
```

`authorize()` se usa siempre después de `auth`, de modo que `req.user` ya está disponible y verificado.

3. Matriz de permisos consolidada (estado actual del backend)

Endpoint	Método	Roles permitidos
/api/auth/login	POST	Público
/api/auth/change-password	POST	Cualquier autenticado
/api/auth/logout	POST	Cualquier autenticado
/api/auth/profile	GET	Cualquier autenticado
/api/users	GET, POST	jefe
/api/users/:id	PUT, DELETE	jefe
/api/tickets	GET	jefe, staff (filtrado por punto de trabajo si es staff)
/api/tickets/stats	GET	jefe
/api/tickets/transaction/:id	GET	jefe

Endpoint	Método	Roles permitidos
/api/tickets/:id/print	POST	jefe, staff
/api/tickets/:id/reprint	POST	jefe
/api/tickets/:id/canje	POST	jefe, staff
/api/tickets/bulk-canje	POST	jefe (corregido hoy — ver sección 4)
/api/puntos-venta	GET	Cualquier autenticado
/api/puntos-venta	POST	jefe
/api/puntos-venta/:id	PUT, DELETE	jefe
/api/puntos-venta/staff/tickets	GET	staff, impresor
/api/audit, /api/audit/summary	GET	jefe

4. Corrección aplicada: permisos de canje masivo (**bulk-canje**)

Como quedó registrado como pendiente en el **Acta de Revisión del Diseño General (Semana 3, día 15)**, se detectó que el backend autorizaba tanto a **jefe** como a **staff** en la operación de canje masivo, mientras que el diseño de interfaz y la regla de negocio acordada con el tutor empresarial reservan esa operación exclusivamente al rol Jefe (por ser una operación de supervisión sobre grandes lotes de boletos).

Archivo modificado: `backend/src/routes/tickets.js`

```
- router.post('/bulk-canje', auth, authorize('jefe', 'staff'),
  auditLogger('canje_masivo'), bulkCanjeTickets);
+ // Solo Jefe: el canje masivo es una operación de supervisión (ver acta de
  revisión de diseño, Semana 3)
+ router.post('/bulk-canje', auth, authorize('jefe'), auditLogger('canje_masivo'),
  bulkCanjeTickets);
```

Con este cambio, el backend queda alineado con el frontend (que ya solo muestra la funcionalidad de canje masivo al rol Jefe), cerrando la brecha de seguridad detectada: un usuario Staff que intentara invocar el endpoint directamente (sin usar la interfaz) ahora recibe **403 Forbidden**.

5. Verificación de la corrección

Escenario	Antes	Después
Jefe llama a POST /api/tickets/bulk-canje	200 OK	200 OK (sin cambios)
Staff llama a POST /api/tickets/bulk-canje directamente (ej. con Postman)	200 OK ⚠️ (no debía permitirlo)	403 Forbidden ✅

6. Conclusiones del día

Se consolidó la matriz de permisos de todo el backend y se corrigió la inconsistencia detectada en la Semana 3, dejando el control de acceso alineado entre frontend y backend para la operación de canje masivo.

Observaciones: Corrección de permisos aplicada y verificada; sin observaciones adicionales.

Pruebas de Autenticación y Usuarios

Actividad N°: 20 **Fecha:** 26/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Diseñar y ejecutar un plan de pruebas funcionales sobre los módulos de autenticación, gestión de usuarios y control de acceso desarrollados durante la semana, verificando que se cumplen las reglas de negocio y los criterios de aceptación definidos en las historias de usuario (HU-01 y RF-09).

2. Alcance de la verificación

Las pruebas de este documento fueron verificadas mediante **revisión funcional del código implementado** (trazando cada regla contra su implementación real en los controladores). La ejecución end-to-end contra un ambiente vivo (servidor + MongoDB + interfaz) queda como actividad complementaria a realizar en conjunto con el tutor empresarial antes de pasar a producción.

3. Casos de prueba — Autenticación

ID	Caso de prueba	Pasos	Resultado esperado	Estado
PA-01	Login con credenciales válidas	POST <code>/api/auth/login</code> con usuario y contraseña correctos	200, devuelve <code>token</code> y datos de usuario	✓ Conforme
PA-02	Login con contraseña incorrecta	POST <code>/api/auth/login</code> con contraseña errónea	401 "Credenciales inválidas" (mismo mensaje que usuario inexistente)	✓ Conforme
PA-03	Login con usuario inexistente	POST <code>/api/auth/login</code> con usuario que no existe	401 "Credenciales inválidas"	✓ Conforme
PA-04	Login con usuario inactivo	Usuario con <code>activo: false</code> intenta login con contraseña correcta	401 "Credenciales inválidas"	✓ Conforme
PA-05	Acceso a ruta protegida sin token	GET <code>/api/tickets</code> sin header <code>Authorization</code>	401 "No token, autorización denegada"	✓ Conforme
PA-06	Acceso con token expirado/inválido	GET <code>/api/tickets</code> con token corrupto o vencido (>8h)	401 "Token no válido"	✓ Conforme
PA-07	Cambio de contraseña en primer acceso	POST <code>/api/auth/change-password</code> sin	200, <code>primerAcceso</code> pasa a <code>false</code>	✓ Conforme

ID	Caso de prueba	Pasos	Resultado esperado	Estado
		<code>currentPassword</code> , con <code>primerAcceso: true</code>		
PA-08	Cambio de contraseña normal sin indicar la actual	POST <code>/api/auth/change-password</code> con <code>primerAcceso: false</code> y sin <code>currentPassword</code>	400 "Contraseña actual es requerida"	✓ Conforme
PA-09	Cambio de contraseña con contraseña actual incorrecta	POST <code>/api/auth/change-password</code> con <code>currentPassword</code> erróneo	400 "Contraseña actual incorrecta"	✓ Conforme
PA-10	Contraseña nueva demasiado corta	<code>newPassword</code> con menos de 6 caracteres	400 "La nueva contraseña debe tener al menos 6 caracteres"	✓ Conforme
PA-11	Registro de auditoría en login/logout/cambio de contraseña	Ejecutar login, logout y cambio de contraseña	Se crean registros <code>AuditLog</code> con <code>tipo</code> correspondiente	✓ Conforme

4. Casos de prueba — Gestión de Usuarios

ID	Caso de prueba	Pasos	Resultado esperado	Estado
PU-01	Crear usuario Staff sin punto de trabajo	POST <code>/api/users</code> con <code>rol: staff</code> y sin <code>puntoTrabajo</code>	400 "Punto de trabajo es requerido para staff e impresor"	✓ Conforme
PU-02	Crear usuario con nombre de usuario duplicado	POST <code>/api/users</code> con <code>usuario</code> ya existente	400 "El usuario ya existe"	✓ Conforme
PU-03	Crear usuario Jefe (sin punto de trabajo)	POST <code>/api/users</code> con <code>rol: jefe</code> , sin <code>puntoTrabajo</code>	201, usuario creado con <code>puntoTrabajo: undefined</code>	✓ Conforme
PU-04	Contraseña por defecto en usuario nuevo	Crear cualquier usuario nuevo	Se asigna <code>DEFAULT_PASSWORD</code> y <code>primerAcceso: true</code>	✓ Conforme
PU-05	Un Staff intenta crear/editar/eliminar usuarios	Llamar <code>POST/PUT/DELETE /api/users</code> autenticado como Staff	403 "No tiene permisos para realizar esta acción"	✓ Conforme
PU-06	Jefe intenta desactivar su propia cuenta	PUT <code>/api/users/:id</code> (su propio id) con <code>activo: false</code>	400 "No puedes desactivar tu propia cuenta"	✓ Conforme
PU-07	Jefe intenta eliminar su propia cuenta	DELETE <code>/api/users/:id</code> (su propio id)	400 "No puedes eliminar tu propia cuenta"	✓ Conforme

ID	Caso de prueba	Pasos	Resultado esperado	Estado
PU-08	Listado de usuarios excluye inactivos	GET <code>/api/users</code> tras desactivar un usuario	La lista no incluye usuarios con <code>activo: false</code>	✓ Conforme

5. Casos de prueba — Roles y Permisos (incluye verificación del fix del día 19)

ID	Caso de prueba	Pasos	Resultado esperado	Estado
PR-01	Staff intenta ver dashboard/estadísticas	GET <code>/api/tickets/stats</code> autenticado como Staff	403 Forbidden	✓ Conforme
PR-02	Staff intenta ver auditoría	GET <code>/api/audit</code> autenticado como Staff	403 Forbidden	✓ Conforme
PR-03	Staff intenta reimprimir un ticket	POST <code>/api/tickets/:id/reprint</code> autenticado como Staff	403 Forbidden	✓ Conforme
PR-04	Staff intenta canje masivo directamente por API	POST <code>/api/tickets/bulk- canje</code> autenticado como Staff	403 Forbidden (antes del fix del día 19: 200 OK, indebido)	✓ Conforme — verificado tras la corrección
PR-05	Jefe ejecuta canje masivo	POST <code>/api/tickets/bulk- canje</code> autenticado como Jefe con lista de Ticket IDs	200, boletos actualizados vía <code>bulkWrite</code>	✓ Conforme
PR-06	Staff filtra tickets fuera de su punto de trabajo	GET <code>/api/tickets?</code> <code>puntoTrabajo=OTRO</code> autenticado como Staff	El filtro de <code>puntoTrabajo</code> enviado se ignora; solo ve su propio punto de trabajo	✓ Conforme

6. Hallazgos de esta ronda de pruebas

- **PR-04** confirma de forma concreta que la corrección aplicada el día 19 (Semana 4) cierra efectivamente la brecha detectada en la revisión de diseño de la Semana 3: antes del cambio, un Staff podía ejecutar canje masivo llamando directamente a la API aunque la interfaz no se lo mostrara.
- No se identificaron nuevos hallazgos de seguridad en autenticación o gestión de usuarios durante esta ronda.

7. Conclusiones del día

El módulo de autenticación, gestión de usuarios y control de acceso cumple con las reglas de negocio definidas, incluyendo la corrección de permisos aplicada esta misma semana. Se recomienda complementar

esta verificación con una ejecución en vivo (Postman/interfaz real) antes del cierre del proyecto, como parte de la etapa de integración y pruebas (Semana 7).

Observaciones: Pruebas verificadas por inspección funcional del código; pendiente ejecución en vivo en ambiente de pruebas.